



Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Análisis Numérico para la Ingeniería - CE1111

Informe Problema 2

Profesor: Juan Pablo Soto Quirós

Alumnos:

Mauro Andrés Navarro Obando

Paula Melina Porras Mora

Grupo: 1

05 marzo 2026

1. Estrategia utilizada para valores de funciones

Los valores iniciales para cada función fueron elegidos basándose en el comportamiento de las funciones y la naturaleza de los métodos utilizados. Para el método de bisección, se seleccionaron intervalos en los que la función mostraba un cambio de signo, garantizando la existencia de una raíz en ese intervalo. En cuanto a los métodos de Newton-Raphson, secante y otros, se eligieron valores iniciales cercanos a las raíces esperadas, lo que ayuda a asegurar que los métodos converjan de manera eficiente. La elección de estos valores se realizó a partir de una combinación de observaciones gráficas y conocimiento previo de las funciones, con el objetivo de asegurar una buena aproximación inicial para los métodos iterativos.

Para encontrar las intersecciones con el eje x , la estrategia fue igualar el numerador a cero, mientras que para la intersección con el eje y se revisa si $x = 0$ pertenece al dominio. Como en este caso $x = 0$ anula el denominador, se concluye que no existe intersección con el eje y .

2. Descripción de métodos

- **Método de bisección:** divide el intervalo en dos partes y busca una raíz en el intervalo donde la función cambia de signo, la función debe ser continua y cambiar de signo en los extremos del intervalo.
- **Método de Newton-Raphson:** usa la derivada de la función para aproximar la raíz con la fórmula $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$, la función debe ser diferenciable y la derivada no debe ser cero.
- **Método de secante:** similar a Newton-Raphson, pero usa dos puntos cercanos en lugar de la derivada para aproximar la raíz, los puntos iniciales deben estar cerca de la raíz y la función debe ser continua.
- **Método de Steffensen:** acelera la convergencia de la secante usando una aproximación cuadrática, la función debe ser de punto fijo.
- **Método de la falsa posición:** similar a bisección, pero usa una línea recta para aproximar la raíz en lugar de dividir el intervalo, la función debe cambiar de signo en los extremos del intervalo.
- **Método de Müller:** usa tres puntos y una parábola para aproximar la raíz, se requiere tres puntos iniciales cercanos a la raíz.

3. Fragmentos relevantes de la implementación

A continuación, se muestran partes del código relevantes para cada uno de los métodos.

```
# Método de Bisección
def biseccion(f, a, b):
    inicio = time.perf_counter()
    if f(a) * f(b) >= 0: return None, None, 0, 0.0, "Intervalo inválido"
    for i in range(MAX_ITER):
        mid = (a + b) / 2.0
        if abs(f(mid)) < TOL: return mid, abs(f(mid)), i, time.perf_counter() - inicio, "OK"
        a, b = (mid, b) if f(a) * f(mid) < 0 else (a, mid)
    return mid, abs(f(mid)), i, time.perf_counter() - inicio, "OK"
```

Figura 1: Este fragmento muestra cómo el método divide el intervalo y evalúa la función en el punto medio en cada iteración. Se busca el cambio de signo en el intervalo para encontrar la raíz.

```
# Método de Newton-Raphson
def newton_raphson(f, df, x0):
    inicio = time.perf_counter()
    for i in range(MAX_ITER):
        fxk, dfxk = f(x0), df(x0)
        if abs(dfxk) < 1e-14:
            print(f"Advertencia: Derivada cercana a cero en x0 = {x0}.")
            return x0, float('nan'), i, time.perf_counter() - inicio, "Derivada cercana a cero"
        x1 = x0 - fxk / dfxk
        if criterio_parada(x0, x1, f(x1)): return x1, abs(f(x1)), i, time.perf_counter() - inicio, "OK"
        x0 = x1
    return x1, float('nan'), i, time.perf_counter() - inicio, "Convergencia no alcanzada"
```

Figura 2: Este fragmento aplica la fórmula de Newton-Raphson para iterar y calcular la nueva aproximación de la raíz. Si la derivada es muy pequeña, emite una advertencia.

```
# Método de la Secante
def secante(f, x0, x1):
    inicio = time.perf_counter()
    for i in range(MAX_ITER):
        fx0, fx1 = f(x0), f(x1)
        if fx1 - fx0 == 0: return None, None, i, time.perf_counter() - inicio, "División por cero"
        x2 = x1 - fx1 * (x1 - x0) / (fx1 - fx0)
        if criterio_parada(x1, x2, f(x2)): return x2, abs(f(x2)), i, time.perf_counter() - inicio, "OK"
        x0, x1 = x1, x2
    return x2, abs(f(x2)), i, time.perf_counter() - inicio, "OK"
```

Figura 3: En este método, se usan dos puntos cercanos para calcular la nueva aproximación de la raíz sin necesidad de derivadas, empleando una fórmula basada en los dos valores anteriores.

```
# Método de Steffensen
def steffensen(f, x0):
    inicio = time.perf_counter()
    for i in range(MAX_ITER):
        fx0 = f(x0)
        fx0_fx0 = f(x0 + fx0)
        denom = fx0_fx0 - fx0
        if abs(denom) < 1e-14: return x0, abs(fx0), i, time.perf_counter() - inicio, "División por cero"
        x1 = x0 - fx0**2 / denom
        if criterio_parada(x0, x1, f(x1)): return x1, abs(f(x1)), i, time.perf_counter() - inicio, "OK"
        x0 = x1
    return x1, abs(f(x1)), i, time.perf_counter() - inicio, "OK"
```

Figura 4: Steffensen es una versión acelerada del método de la secante. Usa una aproximación cuadrática para acelerar la convergencia.

```

# Método de Falsa Posición
def falsa_posicion(f, a, b):
    inicio = time.perf_counter()
    for i in range(MAX_ITER):
        fa, fb = f(a), f(b)
        denom = fb - fa
        if abs(denom) < 1e-14: return None, None, i, time.perf_counter() - inicio, "División por cero"
        xk = a - fa * (b - a) / denom
        fxk = f(xk)
        if criterio_parada(a, xk, fxk): return xk, abs(fxk), i, time.perf_counter() - inicio, "OK"
        a, b = (xk, b) if fa * fxk < 0 else (a, xk)
    return xk, abs(fxk), i, time.perf_counter() - inicio, "OK"

```

Figura 5: El método de falsa posición aproxima la raíz utilizando una línea recta entre los puntos a y b , y actualiza el intervalo de acuerdo con el signo de $f(x_k)$.

```

# Método de Müller
def muller(f, x0, x1, x2):
    inicio = time.perf_counter()
    for i in range(MAX_ITER):
        f0 = f(x0)
        f1_val = f(x1)
        f2_val = f(x2)

        h0, h1 = x1 - x0, x2 - x1
        if abs(h0) < 1e-14 or abs(h1) < 1e-14: return None, None, i, time.perf_counter() - inicio, "Puntos demasiado cercanos"
        delta0, delta1 = (f1_val - f0) / h0, (f2_val - f1_val) / h1
        denom_d = x2 - x0
        if abs(denom_d) < 1e-14: return None, None, i, time.perf_counter() - inicio, "División por cero"
        d = (delta1 - delta0) / denom_d
        b_coef = delta0 + h1 * d
        discriminante = b_coef**2 - 4 * f2_val * d
        if discriminante < 0: return None, None, i, time.perf_counter() - inicio, "Raíz compleja"
        sqrt_disc = math.sqrt(discriminante)
        denom_final = b_coef + sqrt_disc if abs(b_coef + sqrt_disc) >= abs(b_coef - sqrt_disc) else b_coef - sqrt_disc
        if abs(denom_final) < 1e-14: return None, None, i, time.perf_counter() - inicio, "División por cero"
        xk = x2 - 2 * f2_val / denom_final
        if criterio_parada(x2, xk, f(xk)): return xk, abs(f(xk)), i, time.perf_counter() - inicio, "OK"
        x0, x1, x2 = x1, x2, xk
    return xk, abs(f(xk)), i, time.perf_counter() - inicio, "OK"

```

Figura 6: El método de Müller usa tres puntos para aproximar la raíz utilizando una parábola, y luego encuentra la intersección de la parábola con el eje x .

4. Resultados

A continuación, se presentan los resultados obtenidos al aplicar los diferentes métodos de resolución de ecuaciones no lineales, tales como el método de Bisección, Newton-Raphson, Secante, Steffensen, Falsa Posición y Müller. Para cada función, se muestran las soluciones aproximadas, los errores finales, el número de iteraciones necesarias y el tiempo de ejecución.

Función: f1 Intervalo base: [-1.0, 1.0]						
Método	Valores Iniciales	xk	Error	Iter	Tiempo (s)	Estado
Bisección	a=-1.0, b=1.0	-1.0000000000	1.5403e+00	9999	8.967400e-03	OK
Newton-Raphson	x0=-1.0	0.7390851332	0.0000e+00	8	4.300001e-05	OK
Secante	x0=-1.0, x1=1.0	0.7390851332	0.0000e+00	6	2.009999e-05	OK
Steffensen	x0=-1.0	0.7390851332	0.0000e+00	4	5.230001e-05	OK
Falsa Posición	a=-1.0, b=1.0	0.7390851332	4.6407e-13	10	1.740000e-05	OK
Müller	x0=-1.0000, x1=0.0000, x2=1.0	0.7390851332	0.0000e+00	4	2.579999e-05	OK

Función: f2 Intervalo base: [1.0, 2.0]						
Método	Valores Iniciales	xk	Error	Iter	Tiempo (s)	Estado
Bisección	a=1.0, b=2.0	1.5000000000	2.5500e-01	9999	1.634750e-02	OK
Newton-Raphson	x0=1.0	1.4044916482	1.0785e-11	16	4.010000e-05	OK
Secante	x0=1.0, x1=2.0	1.4044916482	4.4400e-16	7	2.040000e-05	OK
Steffensen	x0=1.0	1.4044916482	9.9920e-16	5	1.570000e-05	División por cero
Falsa Posición	a=1.0, b=2.0	1.4044916482	9.0092e-12	13	3.250000e-05	OK
Müller	x0=1.0000, x1=2.5000, x2=2.0	1.4044916482	3.3307e-16	4	2.699999e-05	OK

Figura 7: Resultados de las tablas de comparación de las funciones 1 y 2.

Función: f3 Intervalo base: [-2.0, -1.0]						
Método	Valores Iniciales	xk	Error	Iter	Tiempo (s)	Estado
Bisección	a=-2.0, b=-1.0	-2.000000000	1.0627e+02	9999	3.122420e-02	OK
Newton-Raphson	x0=-2.0	nan	nan	9999	2.872520e-02	Convergencia no alcanzada
Secante	x0=-2.0, x1=-1.0	-1.2076478271	3.5527e-15	7	4.388000e-05	OK
Steffensen	x0=-2.0	-2.000000000	1.0627e+02	9999	4.358200e-02	OK
Falsa Posición	a=-2.0, b=-1.0	-1.2076478271	4.7633e-11	22	8.588001e-05	OK
Müller	x0=-2.0000, x1=-2.5000, x2=-1.0	-1.2076478271	2.6645e-15	6	5.000000e-05	OK

Función: f4 Intervalo base: [-3.0, -1.5]						
Método	Valores Iniciales	xk	Error	Iter	Tiempo (s)	Estado
Bisección	a=-3.0, b=-1.5	-3.000000000	2.8289e+00	9999	1.612260e-02	OK
Newton-Raphson	x0=-3.0	2.0287578381	4.1158e-11	27	6.120000e-05	OK
Secante	x0=-3.0, x1=-1.5	-2.0287578381	3.3307e-16	6	1.920000e-05	OK
Steffensen	x0=-3.0	2.0287578381	3.3307e-16	7	2.350000e-05	División por cero
Falsa Posición	a=-3.0, b=-1.5	-2.0287578381	2.7025e-11	19	4.830000e-05	OK
Müller	x0=-3.0000, x1=-3.7500, x2=-1.5	-2.0287578381	3.3307e-16	5	3.470000e-05	OK

Figura 8: Resultados de las tablas de comparación de las funciones 3 y 4.

Función: f5 Intervalo base: [-1.0, -0.3]						
Método	Valores Iniciales	xk	Error	Iter	Tiempo (s)	Estado
Bisección	a=-1.0, b=-0.3	-0.650000000	3.7161e-01	9999	2.392320e-02	OK
Newton-Raphson	x0=-1.0	nan	nan	9999	2.110610e-02	Convergencia no alcanzada
Secante	x0=-1.0, x1=-0.3	0.000000000	0.0000e+00	6	3.490000e-05	OK
Steffensen	x0=-1.0	0.000000000	2.5411e-21	3	1.240000e-05	División por cero
Falsa Posición	a=-1.0, b=-0.3	0.000000000	4.7535e-13	9	3.280000e-05	OK
Müller	x0=-1.0000, x1=-1.0833, x2=-0.3	-0.000000000	2.4652e-32	6	4.659999e-05	OK

Figura 9: Resultado de la tabla de comparación de la función 5.

Función: f6 Intervalo base: [1.5, 3.0]						
Método	Valores Iniciales	xk	Error	Iter	Tiempo (s)	Estado
Bisección	a=1.5, b=3.0	2.250000000	9.5312e-01	9999	5.515000e-03	OK
Newton-Raphson	x0=1.5	nan	nan	9999	1.094550e-02	Convergencia no alcanzada
Secante	x0=1.5, x1=3.0	2.000000000	0.0000e+00	8	1.160000e-05	OK
Steffensen	x0=1.5	2.000000000	0.0000e+00	175	1.309000e-04	OK
Falsa Posición	a=1.5, b=3.0	2.000000000	2.5798e-11	29	2.160000e-05	OK
Müller	x0=1.5000, x1=3.7500, x2=3.0	N/A	N/A	1	6.200004e-06	Raíz compleja

Función: f7 Intervalo base: [-1.0, 0.5]						
Método	Valores Iniciales	xk	Error	Iter	Tiempo (s)	Estado
Bisección	a=-1.0, b=0.5	N/A	N/A	0	0.000000e+00	Intervalo inválido
Newton-Raphson	x0=-1.0	7418796361.5831158493	nan	9999	1.414820e-02	Convergencia no alcanzada
Secante	x0=-1.0, x1=0.5	0.000000000	0.0000e+00	37	3.830000e-05	OK
Steffensen	x0=-1.0	0.000000000	0.0000e+00	12	6.99991e-06	División por cero
Falsa Posición	a=-1.0, b=0.5	-201.1707675219	2.0000e+00	9999	8.811000e-03	OK
Müller	x0=-1.0000, x1=-0.4167, x2=0.5	N/A	N/A	0	3.399997e-06	Raíz compleja

Figura 10: Resultado de las tablas de comparación de las funciones 6 y 7.